프로젝트 기반 기술 교재 · v1.0

러너 모델링 테스트

MakeHuman에서 웹까지, 모델이 바뀌고 움직이는 과정을 이해하기



실제 로컬 시험 화면 | 허벅지 55cm · 외형과 뼈대를 함께 변경한 결과

이 교재에서 배우는 것

MPFB·Blender의 모델을 GLB로 가져오고, Three.js에서 관절과 길이를 제어합니다. 결과를 화면뿐 아니라 수치로 확인하는 방법까지 다룹니다.

대상: 기본적인 JavaScript·파일/폴더 개념을 아는 입문 개발자. 3D 벡터·행렬은 필요한 부분부터 설명합니다. 그림은 실제 캡처와 설명용 도식을 구분했습니다.

작성일 2026-10-14. 근거는 저장소 코드와 2026-10-04의 기존 검사 기록입니다. 교재 제작 중 모델 실험을 새로 실행한 결과로 표시하지 않습니다.

학습 안내

학습 순서와 도착점

한 장에서 한 가지 원리를 배우고, 해당 코드와 실습으로 연결합니다.

준비
모델·파일 이해
→
표시
웹에서 불러오기
→
제어
자세·길이 변경
→
검증
측정·복원·기록

페이지	학습 범위	읽은 뒤 할 수 있는 일
03-05	기술 구성·3D 데이터·파일	Blender 작업 파일과 웹 모델 구분
06-09	MPFB·내보내기·남녀 모델	원본을 지키며 GLB 준비
10-12	웹 실행·렌더링·로드 관리	모델이 보이는 과정 설명
13-15	좌표·관절 제한·회전	사용자 각도를 뼈 방향에 연결
16-20	크기·길이·스킨 재구성	외형과 관절을 함께 변경하는 원리 설명
21-22	관리자·로컬 저장·통계	시험 설정과 운영 기능 구분
23-26	실습·자동 검사·결과·오류	변경 후 무엇을 검증할지 판단
27-30	미구현 범위·복습·용어·출처	다음 작업을 작은 단위로 검토

교재 표시 규칙
실제 코드: 저장소 발췌. **축약 코드**: 설명용이며 그대로 붙여넣을 완성 함수가 아닙니다. **검토 제안**: 아직 구현되지 않은 기능입니다.

실습은 로컬 시험 화면을 사용합니다. 코드 변경·새 기능 추가는 설명과 검토 후 하나씩 진행한다는 프로젝트 원칙을 유지합니다.

구현 해설

무엇으로 만들었나요?

현재 시험에 실제 사용한 도구와, 서비스 확장 후보를 분리합니다.

Blender + Python
원본 편집·GLB 출력
→
Vite + TypeScript
웹 코드 실행·검사
→
Three.js + WebGL
브라우저 3D 표시

기술	프로젝트에서 맡은 일	버전·상태
MakeHuman / MPFB	인체 형상·변형 데이터, Blender 내 편집	MPFB 2.0.17 시험
Blender / bpy / bmesh	리깅·복사본 정리·자동 내보내기	Blender 5.2.2 LTS 기록
TypeScript	모델/설정 타입, 조절 함수	선언 7.0.2
Three.js	장면·카메라·뼈·메시·행렬	선언 0.186.1
Vite / Node.js	개발 서버·번들·명령 실행	Vite 8.3.2 / 시험 Node 24.19.0
Playwright + Node assert	브라우저 조작·수치·오류 검사	별도 설치 환경
localStorage / IndexedDB	설정·이벤트 / GLB 파일 보관	로컬 시험만 사용

아직 이 시험에 쓰지 않은 기술
React·R3F·Drei 전환, Python 계산 API, Supabase 운영 인증/통계는 설계 후보 또는 연결 전입니다.
Blender 자동화에 Python을 썼다는 사실과 기록 계산 서버 구현은 다릅니다.

근거: experiments/model-preview/package.json

구현 해설

모델은 무엇으로 이루어질까요?

외형, 뼈대, 재질, 동작은 각각 다른 데이터입니다.




```
P_new = 0.7 * T_A(P) + 0.3 * T_B(P)
```

개념도 | 실제 가중치는 GLB의 skinWeight에서 읽습니다.

데이터	역할	이번 시험에서 볼 곳
Mesh / Geometry	정점과 면으로 몸의 표면 구성	geometry.attributes.position
Normal / Material	빛에 반응하는 방향 / 표면 표현	매끈한 법선·단색 재질
Bone / Skeleton	관절의 계층 구조와 연결	53개 뼈 · SkeletonHelper
Skin indices / weights	각 정점에 영향을 주는 뼈 번호와 비율	skinIndex · skinWeight
Morph target / Shape key	기본 정점에 더하는 외형 변형	내부 키 ≠ cm·kg 입력
Animation clip	시간에 따른 변화 데이터	현재 내보내기에서는 꺼짐

뼈가 있다는 것만으로 피부가 따라오지는 않습니다. 정점과 뼈를 연결하는 가중치가 필요합니다. 반대로 변형키로 표면만 바꾸면 관절 중심은 자동으로 이동하지 않을 수 있습니다.

기본 개념 참고: 공식 Three.js SkinnedMesh [S2]. 실제 데이터 구조는 프로젝트의 불러오기 코드로 확인합니다.

구현 해설

파일이 지나가는 길

원본 보존과 웹 전달을 분리하면 시행착오를 되돌릴 수 있습니다.

원본 .blend
편집·리깅·변형
→
정리된 복사본
보조 형상 제거
→
단일 .glb
웹 전달 모델
→
검증 .json
해시·개수·결과

파일	역할	주의
base.obj	hm08 기본 정점·면과 그룹 비교 기준	완성된 웹 러너 파일이 아님
runner-test.blend	사용자가 저장한 편집 원본	원본을 덮어쓰지 않음
runner-*.glb	불러올 형상·재질·스킨·변형 데이터	MPFB의 모든 UI 기능이 따라오지는 않음
runner-*.export.json	내보내기 개수·제약·SHA-256	모델 자체를 대신하지 않음
package-lock.json	웹 의존성의 고정된 설치 정보	npm ci로 재현
검증 JSON	시험 환경·오차·검사 항목	당시 시험 범위의 증거

저장소 루트 기준 경로

```
data/processed/makehuman/runner-test.blend
experiments/model-preview/public/models/runner-male.glb
experiments/model-preview/public/models/runner-female.glb
docs/05-validation/reports/
```

다른 PC에서 열 때
Git에는 원본 .blend와 GLB가 들어 있지 않습니다. 필요한 파일을 같은 상대 위치에 별도로 복사하거나, 원본과 MPFB 환경을 준비해 다시 변환해야 합니다.

구현 해설

MPFB와 리깅 준비

설치부터 반복하지 말고, 이미 저장한 원본이 있으면 상태 확인부터 시작합니다.

MPFB 활성화
Blender 확장
→
Human 생성
또는 원본 열기
→
뼈대 연결
리깅·움직임 확인
→
원본 저장
기본 자세 보관

단계	행동	확인
설치	Edit → Preferences → Get Extensions → Install from Disk	프로젝트 기록은 MPFB 2.0.17 ZIP
도구 열기	3D View에서 N → MPFB 탭	기본 사람 생성 또는 기존 .blend 열기
구조	Outliner의 Human과 Human.rig 확인	Human은 표면, Human.rig는 Armature
동작	Armature 선택 → Pose Mode에서 뼈 시험	관절을 움직이면 주변 표면이 따라옴
다시 맞춤	체형 변경 후 원본 MPFB에서 뼈대/자산 재맞춤 확인	웹으로 보내기 전에 관절 정렬 검사
저장	기본 자세로 돌리고 .blend 저장	자동 변환은 중립 자세를 요구

피부와 옷이 없는 이유
이번 단계는 외형 전달과 관절 변형을 검사하는 단계입니다. 피부 텍스처·복장은 별도 자산이며, 먼저 추가하면 문제의 원인을 구분하기 어려워집니다.

메뉴는 프로젝트의 Blender 5.2.2·MPFB 2.0.17 안내 기준입니다. 확장 버전마다 배치가 달라질 수 있습니다. 공식 설치 안내 [S4]와 프로젝트 가이드를 함께 확인하세요.

근거: docs/06-guides/MPFB_TO_GLB_GUIDE.md

구현 해설

Blender에서 GLB로 내보내기

복사본에서 보조 형상을 정리하고, 몸체와 연결된 뼈대를 함께 선택합니다.

Export copy
원본과 분리
→
몸체 + 뼈대
복사본만 선택
→
glTF 2.0
GLB 형식
→
웹에서 확인
표시 → 조절 → 복원

위치	시험 설정	이유
MPFB → Operations → Export copy	Bake modelling shapekeys 끄기	변형키를 복사본에 보존
Export copy	Mask/Subdiv: Remove · Delete helpers 켜기	시험용 표면만 남기기
Export copy	Remove basemesh 끄기	몸체 자체는 보존
File → Export → glTF 2.0	glTF Binary (.glb) · Selected Objects	선택한 복사본만 한 파일로 출력
Data	Apply Modifiers 끄기 · Shape Keys/Skinning 켜기	이번 변형·리깅 전달 조건
Animation	끄기	현재 달리기 동작은 없음

치마나 긴 덮개처럼 보이는 형상이 실제 의류가 아니라 내부 작업용 보조 형상일 수 있습니다. 원본을 무작정 지우기보다 내보낼 복사본을 정리합니다.

수동 출력과 웹 허용 목록
새로 내보낸 GLB는 해시가 달라질 수 있습니다. 현재 관리자는 미확인 파일을 표시 시험만 허용하며, 관절 방향 확인 없이 사용자 모델로 게시하지 않습니다.

Export copy의 복사·변형키·helpers 개념: MPFB 공식 문서 [S5]. 메뉴 설정의 근거는 프로젝트 가이드입니다.

구현 해설

Python 자동 변환의 안전장치

모델 정점 번호에 의존하는 정리는 원본 구조가 같을 때만 안전합니다.



실제 코드 | export_runner.py:92-99

```
result = bpy.ops.export_scene.gltf(
    filepath=str(output), export_format='GLB', use_selection=True,
    export_apply=False, export_skins=True, export_morph=True,
    export_morph_normal=True, export_animations=False,
    export_materials='EXPORT', export_rest_position_armature=True,
    export_cameras=False, export_lights=False,
)
assert result == {'FINISHED'} and output.is_file()
```

코드의 검사	왜 필요한가
Human 존재·Mesh 여부·연결 Armature	몸체와 뼈대가 빠진 출력 방지
hm08 정점/면 수·body 면 정점 비교	다른 토폴로지에서 잘못된 정점 삭제 방지
중립 자세·허용 Modifier 확인	이미 구부린 상태나 예상 밖 수정자를 차단
Shape key 개수·값·정점 수 유지	정리 과정에서 변형 데이터 손상 확인
파일 존재·SHA-256 기록	어떤 원본에서 무엇을 출력했는지 추적

이 스크립트는 저장된 hm08 모델 전용입니다. 임의의 모델·리토폴로지·의상 모델을 범용으로 정리하는 도구가 아닙니다. 지원하지 않는 구조에서는 중단하도록 만들었습니다.

근거: experiments/model-preview/export_runner.py

구현 해설

남성형·여성형을 따로 만드는 이유

외형을 바꾼 뒤 뼈대도 맞춰야 같은 관절 조절이 작동합니다.

원본 다시 열기
모델별 독립 시작
→
gender 적용
MPFB 외형 계산
→
rig refit
관절 위치 재맞춤
→
별도 GLB
해시·기록 분리

실제 호출 발췌 | export_variants.py

```
HumanObjectProperties.set_value('gender', gender,
    entity_reference=body)
TargetService.reapply_macro_details(body)
RigService.refit_existing_armature(rig, body)
bpy.context.view_layer.update()
```

출력	정점 / 면 / 뼈	파일 크기
runner-male.glb	13,380 / 13,378 / 53	2.66 MiB
runner-female.glb	13,380 / 13,378 / 53	2.65 MiB

값을 바꾼 뒤 단순히 내보내기만 하지 않고, TargetService로 외형을 다시 적용한 다음 RigService로 Armature를 맞춥니다. 반복마다 원본을 다시 열므로 앞 모델의 편집이 다음 모델에 누적되지 않습니다.

원본 보존 검사
변환 전후 원본 .blend의 SHA-256을 비교합니다. 두 GLB의 존재만으로 원본이 보존됐다고 판단하지 않습니다. 파일명 변경만으로 모델 검증을 대신하지도 않습니다.

근거: experiments/model-preview/export_variants.py

구현 해설

웹 시험 환경을 실행하기

이 실습은 React 없는 TypeScript·Three.js·Vite 프로젝트입니다.

프로젝트 루트에서 시작 | 의존성 설치 후 개발 서버 실행

```
cd experiments/model-preview
npm ci
npm run dev
```

구성	역할
Node.js / npm	웹 개발 도구 실행·의존성 설치
Vite 개발 서버	소스 모듈 제공·변경 내용 반영
TypeScript 검사	타입 불일치 확인 · 실제 3D 정확도 검사는 별도
Vite build	배포용 정적 파일 생성
public/models/	브라우저에서 /models/ 경로로 모델 로드

TypeScript 검사 + 배포용 번들 생성

```
npm run build
```

package.json의 Node 허용 범위는 ^20.19.0 또는 ≥22.12.0입니다. 기존 시험은 24.19.0을 사용했습니다. 다른 PC에서는 설치된 Node와 실제 지원 환경을 확인하세요.

서버 주소와 파일 확인
개발 서버에 출력된 로컬 주소로 접속합니다. 기존 검증 코드는 127.0.0.1:5173을 사용하므로 해당 포트를 확인합니다. 기본 GLB가 없으면 파일 로드가 실패합니다.

근거: docs/06-guides/MODEL_PREVIEW.md

구현 해설

Three.js의 화면 구성

장면에 모델을 넣고 카메라로 본 모습을 렌더러가 캔버스에 그립니다.

Scene
모델·빛·바닥
→
Camera
보는 위치·시야
→
Renderer
WebGL 캔버스
→
Frame loop
매 프레임 표시

실제 구성의 핵심 발췌 | main.ts

```
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(32, 1, .01, 1000);
const renderer = new THREE.WebGLRenderer({ antialias: true });
const controls = new OrbitControls(camera, renderer.domElement);
renderer.setAnimationLoop(() => {
  controls.update();
  renderer.render(scene, camera);
});
```

장치	이유
OrbitControls	마우스 회전·확대·이동 · 관절 입력과 별개
Hemisphere / Directional light	부피와 굴곡을 읽기 쉬운 조명
GridHelper / SkeletonHelper	바닥 기준과 관절 연결을 시각화
ResizeObserver	뷰포트 크기에 맞춰 화면 비율·렌더 크기 갱신
devicePixelRatio 상한 2	고해상도 화면의 픽셀 수를 제한

렌더링 반복과 애니메이션은 다릅니다
setAnimationLoop는 화면을 계속 그립니다. 매 프레임 관절값이 바뀌지 않으면 모델은 같은 자세입니다. 이 코드만으로 달리기가 만들어지지 않습니다.

근거: experiments/model-preview/src/main.ts

구현 해설

GLB 불러오기와 교체

비동기 로딩은 늦게 도착한 이전 결과와 자원 정리까지 고려합니다.



설명용 축약 | main.ts 로딩·요청 순서 검사

```
const gltf = await new GLTFLoader().parseAsync(buffer, '');
const digest = await crypto.subtle.digest('SHA-256', buffer);
if (currentId !== loadId) {
  dispose(gltf.scene);
  return;
}
model = gltf.scene;
scene.add(model);
```

처리	프로젝트 이유
model.traverse	Bone·SkinnedMesh·Morph 정보를 수집
supportedHashes	확인한 모델에서만 자세·길이 조절 활성화
loadId / selectionId	빠른 모델 교체 시 이전 로드 결과 반영 방지
geometry/material/texture.dispose	교체 후 GPU 자원 해제
ImageBitmap.close	사용을 마친 이미지 비트맵 자원 정리
Box3 기반 frame()	모델 크기에 맞춘 카메라 거리·타깃 설정

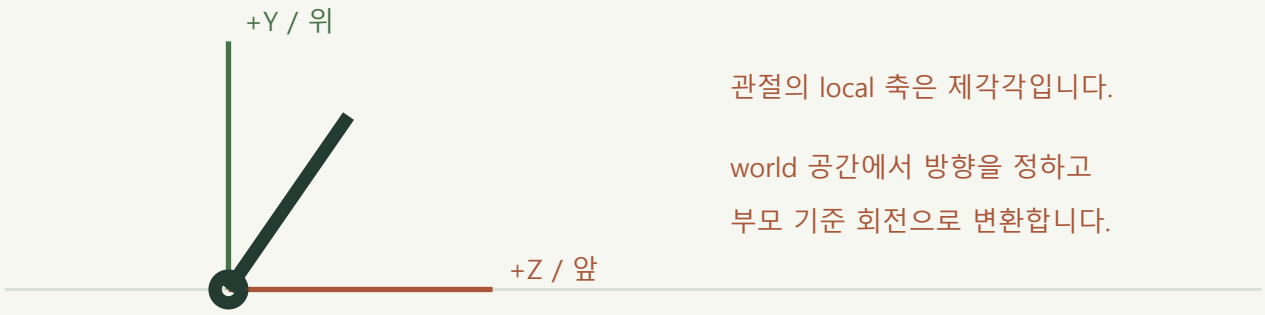
해시는 파일 식별 수단입니다. 같은 뼈 이름을 가진 다른 파일의 관절축·스킨 품질까지 검증하는 도구는 아닙니다. 지원 목록 확대는 별도 모델 검사 후 진행합니다.

GLTFLoader의 비동기 파싱·반환 scene/animations 개념은 공식 문서 [S1], 실제 분기와 정리는 main.ts를 따릅니다.

구현 해설

local과 world 좌표 이해하기

숫자 X·Y·Z를 직접 돌리는 것과 사용자가 원하는 앞뒤 움직임은 다릅니다.



관절의 local 축은 제각각입니다.

world 공간에서 방향을 정하고
부모 기준 회전으로 변환합니다.

개념도 | 위치·방향의 원리를 설명하며 실제 모델 비율을 나타내지 않습니다.

기준	이번 모델의 해석
world +Y	위쪽
world +Z	모델이 바라보는 앞쪽
local 좌표	부모 뼈를 기준으로 한 자식 뼈의 좌표
world 방향	뼈의 현재 위치에서 자식 관절까지의 방향
기준 자세	초기 회전을 저장하고 매번 되돌린 뒤 새 자세 계산

회전 변환의 개념 | 실제 구현은 다음 장에서 확인

```
direction = normalize(childWorld - parentWorld)
localRotation = inverse(parentWorldRotation) * targetWorldRotation
```

왜 이전 머리 90° 화면이 문제였나요?
모든 뼈에 동일한 X/Y/Z 슬라이더를 제공하면 모델 내부 축이 그대로 사용자 입력이 됩니다. 러닝에 필요한 관절만 고르고 의미 있는 방향·각도로 제한해야 합니다.

근거: experiments/model-preview/src/running-pose.ts

구현 해설

러닝 자세 입력과 제한

현재는 좌우 4개 관절 종류, 총 8개 자세 입력을 제공합니다.

부위	의미	기본 / 허용 범위
어깨	팔 앞뒤 · 아래로 향하면 0°	0° / -25° ~ 45°
팔꿈치	앞으로 굽힘 · 편 팔이 0°	80° / 45° ~ 120°
고관절	다리 앞뒤 · 아래로 향하면 0°	0° / -15° ~ 65°
무릎	뒤로 굽힘 · 편 다리가 0°	5° / 0° ~ 120°

숫자 입력
유한값 검사
→
관절 정책
노출·최소·최대
→
clamp + round
허용 각도로 보정
→
apply()
기준부터 재계산

설명용 수식 | angle은 실제 코드에서 rad로 변환

```
s = sqrt(1 - lateral * lateral)
d = (lateral, -cos(angle) * s, sin(angle) * s)

// fixed outward directions in the current implementation
armLateral = sign * sin(12 degrees)
legLateral = sign * sin(5 degrees)
```

다리가 벌어져 보이는 이유 중 하나는 바깥 방향 5°가 고정됐기 때문입니다. 골반 너비는 두 고관절의 시작 간격이고, 벌림은 그 지점에서 다리가 나가는 방향입니다.

이 제한의 의미
모든 사람의 해부학적 가동범위를 검증한 값이 아닙니다. 신체끼리 관통하지 않는지, 발이 바닥에 붙는지는 별도 검사입니다. 관리자도 현재 상한 안에서만 범위를 좁힙니다.

구현 해설

Quaternion으로 방향 맞추기

뼈의 현재 방향을 목표 방향으로 옮긴 뒤 부모 기준의 회전값으로 바꿉니다.

현재 방향
자식 위치 - 뼈 위치
→
회전 차이
현재 → 목표
→
world 회전
차이 × 현재
→
local 회전
부모 역회전 적용

실제 코드 | running-pose.ts:31-36

```
const current = p(child).sub(p(name)).normalize();
const world = b.getWorldQuaternion(new THREE.Quaternion());
const parent = b.parent!.getWorldQuaternion(new THREE.Quaternion());
const delta = new THREE.Quaternion().setFromUnitVectors(current,direction);
b.quaternion.copy(parent.invert().multiply(delta).multiply(world)).normalize();
model.updateMatrixWorld(true);
```

setFromUnitVectors는 정규화된 두 방향 사이의 회전을 만듭니다. 이 회전을 현재 world 회전에 곱한 뒤, 부모 world 회전의 역을 앞에 곱합니다. 마지막 normalize는 단위 Quaternion을 유지합니다. [S3]

팔꿈치·무릎의 굽힘 방향

flex()의 개념식 | d와 직교하는 굽힘 방향을 사용

```
bend = normalize(forward - dot(forward, d) * d)
result = normalize(d * cos(a) + bend * sin(a))
```

위팔/허벅지 방향 d에 수직인 굽힘 방향을 만들고, 그 평면에서 a만큼 굽힙니다. 팔은 앞쪽, 무릎은 뒤쪽 기준을 사용합니다. 이 방식은 현재 모델·허용 범위에 맞춘 구현이며 범용 리깅 해법은 아닙니다.

반복 조작에서 지켜야 할 점
현재 회전에 작은 값을 계속 더하면 누적 오차와 이전 입력의 영향이 남습니다. apply()는 저장된 초기 Quaternion부터 복원해 전체 입력을 다시 계산합니다.

구현 해설

전체 키와 부위 길이는 다릅니다

전체 배율은 모든 길이를 함께 바꾸고, 부위 편집은 비율을 바꿉니다.

전체 크기
모든 길이 × 같은 배율

→

부위 길이
선택한 구간만 변경

→

자세
관절의 각도 변경

resizeBody()의 설명용 계산식

```
factor = heightCm / (baseHeight * 100)
model.scale = baseScale * factor
model.position = basePosition
model.position.y += (1 - factor) * (baseFloor - basePosition.y)
```

행동	유지할 것	바뀌는 것
전체 크기 조절	신체 비율·관절 각도	모든 길이·모델 높이
허벅지만 길게	다른 구간 길이·관절 각도·전체 배율	무릎/발 위치·기준 높이
부위 길이 초기화	현재 자세·전체 배율	모든 부위 비율 → 기본값
전체 배율 초기화	부위별 비율·자세	전체 배율 → 1배

baseHeight는 기준 자세의 외형 경계 높이입니다. 사람이 줄자로 측정한 키와 같은 정확도를 보장하지 않습니다. 현재 UI 전체 배율 범위는 기준 높이의 약 ±10%입니다.

발 높이 보정의 범위
전체 확대·축소 때 저장해 둔 바닥 기준이 같이 떠오르지 않도록 위치를 보정합니다. 달리는 동안 발 접촉을 해결하는 자동 접지 기능과는 다릅니다.

근거: experiments/model-preview/src/main.ts · resizeBody / restoreShape

구현 해설

cm는 어디에서 측정하나요?

길이의 의미를 정하지 않으면 숫자만 맞는 것처럼 보일 수 있습니다.



실제 시험 화면 | 허벅지: 고관절 중심에서 무릎 관절 중심까지 55cm

입력	측정 기준
위팔 / 아래팔	어깨→팔꿈치 / 팔꿈치→손목
허벅지 / 종아리	고관절→무릎 / 무릎→발목
몸통	골반→목 시작점까지 관절 연결 구간 길이의 합
어깨너비	왼쪽·오른쪽 어깨 관절 중심 사이

팔다리 길이는 좌우 거리의 평균을 cm로 표시하며, 두 쪽을 같은 비율로 조절합니다. 모델의 world 거리 ×100을 cm로 해석하므로 모델 단위가 달라지면 먼저 정규화가 필요합니다.

현재 입력 비율 | 기본 길이의 ±15% 시험 범위

```
ratio = clamp(inputCm / (baselineCm * globalScale), 0.85, 1.15)
```

근거: experiments/model-preview/src/body-dimensions.ts · measure / set

구현 해설

길이 변경의 전체 순서

이미 구부린 뼈를 늘리는 대신, 기준 형상에서 다시 구성합니다.

원본 복원
정점·뼈 위치·회전
→
관절 이동
선택 구간 길이
→
외형 변형
가중치 따라 변경
→
자세 재적용
기존 값·배율 유지

단계	실제 코드에서 하는 일	왜 필요한가
1. 기준 저장	rest Map, 원본 world 정점, 모델 위치·배율	누적 변경·복원 오차 방지
2. 자식 위치 변경	lowerarm/hand/calf/foot의 local 위치 배율	해당 뼈 구간의 길이를 변경
3. 몸통·어깨	척추 연결 길이 조절 · 어깨 중심 간 목표 거리	분기된 관절 구조까지 함께 고려
4. 표면 재구성	뼈별 변환을 정점 가중치로 혼합	피부와 관절이 떨어지지 않도록 연결
5. 바인딩 갱신	inverse bind·normals·bounds 갱신	새 기준 형상에서 다시 자세 적용
6. 표시 복원	runningPose.apply → 전체 배율 재적용	사용자가 정한 각도 유지

뼈를 scale로만 늘리면 자식 뼈의 크기와 변환에도 영향이 전파됩니다. 현재 구현은 저장한 기준 위치로 돌아가 자식 관절 위치와 표면을 함께 재계산하는 방식을 사용합니다.

실습에서 확인할 불변 조건
허벅지 길이를 바꾼 뒤 팔 길이·종아리 길이·관절 각도를 다시 측정합니다. 화면이 그럴듯해 보여도 이 값들이 함께 달라졌다면 의도한 편집이 아닐 수 있습니다.

근거: experiments/model-preview/src/body-dimensions.ts · rebuild

구현 해설

표면은 어떻게 함께 늘어날까요?

기준 정점을 뼈별로 변형하고, 기존 스킨 가중치로 결과를 섞습니다.



$$P_{new} = 0.7 * T_A(P) + 0.3 * T_B(P)$$

개념도 | 실제 가중치는 GLB의 skinWeight에서 읽습니다.

body-dimensions.ts의 연산을 정리한 식

```
S = I + ((to - from) * transpose(from)) / dot(from, from)
T_bone = newBoneWorld * S * inverse(oldBoneWorld)
p_world_new = sum(weight_i * T_i * p_world_original)
p_local_new = inverse(meshWorld) * p_world_new
```

기호	의미
from / to	기준/변경 후 자식 관절의 local 위치
S	해당 연결 방향을 늘리는 3×3 성분을 포함한 4×4 변환
T_bone	기준 뼈 공간 → 변경된 뼈 공간의 world 변환
weight_i	GLB의 skinWeight · 이 구현은 정점당 4개 슬롯 순회
p_world_original	처음 불러올 때 스킨/변형이 적용된 정점 위치

S는 from 벡터를 to 벡터로 보내는 보정입니다. 실제 코드에서는 from의 길이가 0에 가까우면 나눗셈을 하지 않습니다. 연결 정보가 없는 뼈는 S=I로 두고 새 관절 이동을 따릅니다.

이 식이 보장하지 않는 것
근육 부피 보존·의류 변형·관절 주변 피부의 해부학적 정확도까지 계산하지 않습니다. 제한된 모델의 길이 미리보기이며, 극단값·조합의 표면 품질을 따로 봐야 합니다.

구현 해설

다시 바인딩하고 마무리하기

기준 형상이 바뀌면 뼈-표면 연결 기준과 그리기용 정보도 갱신합니다.

정점 변경
position 갱신
→
법선·경계
표면·크기 재계산
→
inverse bind
새 기준 뼈대
→
pose 재적용
기존 각도 유지

실제 호출 발췌 | 세부 처리 일부 생략

```
geometry.morphAttributes = {};
mesh.morphTargetInfluences = [];
geometry.computeVertexNormals();
geometry.computeBoundingBox();
geometry.computeBoundingSphere();
mesh.skeleton.calculateInverses();
mesh.bind(mesh.skeleton, mesh.matrixWorld);
mesh.skeleton.update();
```

현재 코드는 원래 변형을 정점에 굳혀 반영한 뒤 morph 정보를 제거합니다. 그대로 남기면 같은 외형 변화가 두 번 적용될 수 있기 때문입니다. 이 선택은 얼굴·가슴 변형 슬라이더를 나중에 결합할 때 다시 설계할 지점입니다.

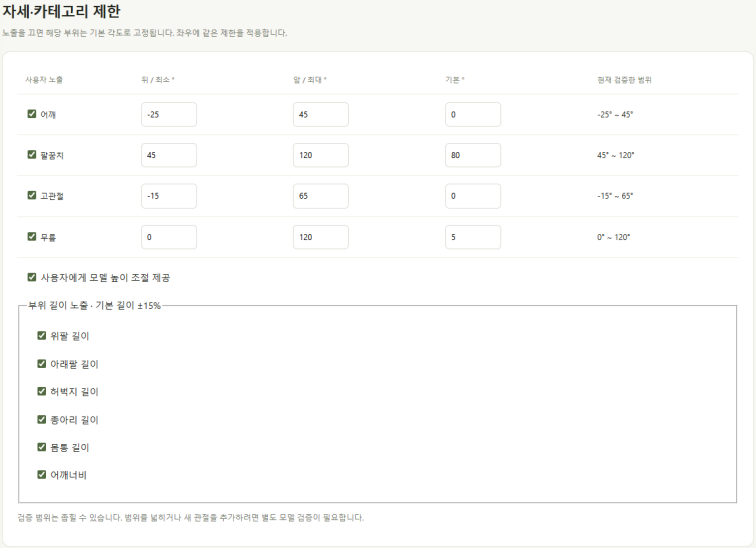
마무리	효과·한계
UV 경계 법선 평균	같은 원본 위치의 중복 정점에 매끈한 법선을 공유
inverse bind 재계산	변경된 기준 뼈대에서 자세 변형 시작
경계 재계산	바뀐 모델 크기를 화면 맞춤 등에 반영
기준 바닥 위치 복원	정적인 발 높이 기준 유지 · 자동 접지는 아님

SkinnedMesh의 bind·경계 정보 개념은 [S2]. 실제 재구성 순서와 단순화는 이 프로젝트 코드의 선택입니다.

구현 해설

관리자에서 제한을 관리하기

사용자가 볼 수 있는 항목과 사용할 수 있는 모델을 정책으로 분리합니다.



실제 로컬 관리자 화면 | 관절 범위와 부위 길이 노출 설정

초안 설정
노출·최소·최대
→
validateSettings
값·범위 검사
→
revision 확인
다른 수정 감지
→
저장
새로 연 화면 반영

정책	현재 동작
관절 숨김	기본 각도로 고정 · 사용자 목록에서 제외
길이 숨김	기본 비율 사용 · 함수에서도 조절 거부
모델 선택	확인한 유형의 기본/등록 모델 사용
동시 수정	저장 시 현재 revision과 초안 revision 비교
관리자 시험	iframe 미리보기 · 통계에서 제외

로컬 제한과 운영 인증
개발 모드이면서 localhost인 경우에만 관리 기능이 작동합니다. 배포본 차단은 실제 관리자 로그인·서버 권한 검사를 대신하지 않습니다.

근거: experiments/model-preview/src/settings.ts

구현 해설

파일·설정·사용 기록을 나누기

로컬 시험 데이터가 무엇을 의미하는지 집계 기준까지 함께 정합니다.

IndexedDB
GLB Blob·파일 정보
→
localStorage
설정·이벤트
→
집계 화면
기간·횟수·순위

데이터	현재 기준
모델 업로드	최대 30MiB · GLB 2.0 헤더/길이 검사 · SHA-256
설정 적용	이 설정 적용 버튼을 눌렀을 때 기록
자세 순위	모델 해시 + 설정 버전 + 전체 관절값 조합
체형 순위	모델/설정/변형 방식 + 높이 5cm·부위 길이 1cm 구간
방문·접속	익명 브라우저 ID · 화면 열기 / 최근 90초 신호
보관·제외	최근 2,000 이벤트 · 관리자 시험·단순 조작 제외

IndexedDB는 파일처럼 큰 구조화 데이터를 비동기로 보관하는 데 사용합니다. localStorage는 이 시험의 작은 설정·이벤트 JSON을 저장합니다. 두 저장소 모두 다른 브라우저·기기와 자동 공유되지 않습니다. [S6]

설명용 축약 | 실제 이벤트는 모델명·높이도 기록

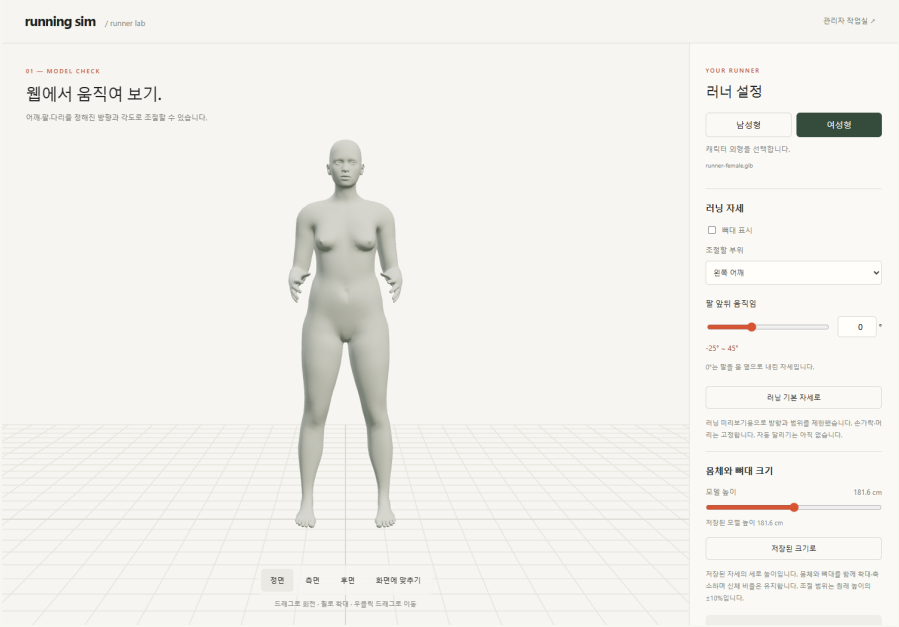
```
record({
  kind: 'configuration_applied',
  revision: settings.revision, hash: modelHash,
  bodyRecipe, lengthsCm: bodyDimensions?.measure(),
  pose: { ...runningPose.values }
});
```

이 숫자의 의미
방문 브라우저 수를 사람 수로, 모델 외형 선택을 사용자의 성별로 해석하지 않습니다. 현재 CSV는 로컬 기록이며 전체 서비스 통계가 아닙니다.

재현 실습 · 코드 수정 없음

화면에서 직접 확인하기

한 번에 한 항목만 바꾸고, 기대한 결과를 적은 뒤 관찰합니다.



기존 여성형 선택 화면 | 당시 캡처이며 현재 길이 항목은 추가되어 있습니다.

순서	조작	확인할 결과
1	남성형 선택 → 뼈대 표시	전신과 관절 연결 표시
2	어깨 20° → 무릎 60°	각 부위의 정해진 방향으로 이동
3	허벅지 기본값 기록 → 약 10% 증가	관절과 표면 동시 변경 · 기존 각도 유지
4	종아리·팔 길이 재확인	바꾸지 않은 구간 길이 유지
5	전체 크기 변경	현재 비율 유지 · 전체 길이 같은 배율
6	부위 길이 초기화 → 배율 초기화	각 초기화의 범위가 서로 다름
7	여성형으로 바꿔 2-6 반복	두 모델 모두 확인
8	이 설정 적용 → 관리자 통계	시험 브라우저에 적용 기록 추가

관찰표에는 모델 유형·기준 길이·입력 길이·실제 길이·각도·이상 현상을 기록합니다. 원하는 길이가 ±15% 범위를 넘으면 현재 시험에서는 제한값으로 보정됩니다.

검사 코드 해설 · 실행은 선택 실습

화면을 수치로 검사하기

브라우저 조작과 관절·정점 측정을 함께 사용해야 회귀를 잡을 수 있습니다.

Playwright
UI 입력·모델 전환
→
page.evaluate
관절·정점 측정
→
Node assert
기대값과 비교
→
JSON + 이미지
결과 기록

별도 터미널 | 저장소 루트에서 실행

```
node experiments/model-preview/tests/body-dimensions.cjs
```

개발 서버 127.0.0.1:5173, 기본 GLB 두 개, Microsoft Edge와 Playwright가 필요합니다. Playwright는 현재 앱 package.json의 의존성이 아닙니다. 별도 설치가 있다면 PLAYWRIGHT_PATH에 모듈 경로를 지정합니다.

검사 의도를 보여주는 축약 | 길이 cm, 각도 °, 복원 world 단위

```
assert(Math.abs(actualCm - targetCm) < 0.0001);
assert.deepEqual(after.pose, before.pose);
assert(after.angleError < 0.0001);
assert(after.points.every(Number.isFinite));
assert(resetError < 0.00001);
```

검사	뜻
입력 길이 / 다른 길이	목표값 일치 + 비대상 구간 불변
각도 / 유한값	기존 자세 유지 + NaN/Infinity 없음
표면 이동 / 복원	외형도 변경 + 기준으로 돌아감
관리자 숨김 / 통계	노출 정책·사용 데이터 분류 유지

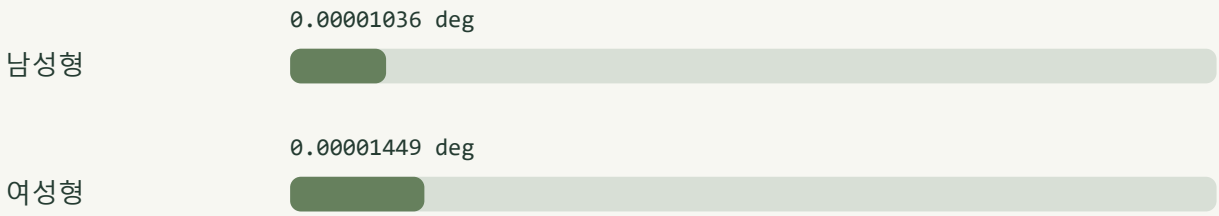
실행 전 알아둘 점
이 스크립트는 별도 브라우저 컨텍스트를 만들지만, 저장소의 검증 JSON과 outputs/ 캡처는 새 결과로 덮어씁니다. 과거 결과와 비교하려면 먼저 사본을 보관합니다. 이 교재 제작에서는 재실행하지 않았습니다.

근거: experiments/model-preview/tests/body-dimensions.cjs

2026-10-04 기존 시험 기록

어디까지 확인했나요?

작은 숫자 오차가 실제 사람이나 러닝 기록의 정확도를 뜻하지는 않습니다.



가로축 최대 = 검사 기준 0.0001° | 기록된 전체 최대 길이 조합의 각도 오차

검사	기록된 결과
단일 부위 양 끝값	남성형·여성형 × 6항목 × 2끝값 = 24건
길이·다른 구간	요청값 오차 <0.0001cm · 비대상 구간 유지 검사
기존 자세	조합 변경 뒤 각도 오차 <0.0001° 검사
원형 복원	남성형 1.526e-07 / 여성형 1.736e-07 world 단위
정점 표본	현재 길이 검사에서 13번째마다 측정
추가 조합	전 항목 최대 · 전체 배율 후 길이 변경
관리·통계	숨긴 길이 함수 거부 · 같은 키/다른 길이 구분

위 수치는 프로그램이 자신이 정의한 관절 길이와 각도를 유지하는지 본 결과입니다. 완전한 표면 충돌 검사, Blender와 모든 정점의 일치, 실제 신체 치수·의학적 가동범위·기록 예측 검증은 포함하지 않습니다.

완료라고 부를 수 있는 범위
지정 GLB 두 개와 시험 환경에서의 길이·관절 제어입니다. 자동 달리기·접지·새 모델 전체·모바일 성능까지 통과했다고 확대하지 않습니다.

근거: docs/05-validation/reports/body-dimensions-verification.json

구현 해설

증상에서 원인으로 좁히기

한 번에 여러 기능을 바꾸지 말고, 데이터·연결·표시 순서로 살펴봅니다.

파일
경로·헤더·해시
→
구조
뼈·가중치·축
→
조절
원본·측정값
→
표시
재질·법선·카메라

증상	먼저 확인	현재 대응
모델이 안 보임	GLB 경로·서버 응답·브라우저 오류	기본 파일 존재 확인 → 화면에 맞추기
치마 같은 덮개	MPFB helper 형상인지 확인	원본 보관 → export copy 정리
피부·옷이 없음	실제 포함한 재질/자산	이번 시험은 단색·복장 전 상태
뼈만 움직임	스킨 연결·가중치·bind 기준	SkinnedMesh와 Armature 연결 확인
길이만 바뀌고 관절이 남음	morph만 변경했는지	길이/뼈대 재구성 경로 확인
다리가 벌어짐	골반 너비와 고정 벌림 5° 구분	현재 간격 입력 없음 · 후속 검토
새 GLB의 입력 잠금	해시·지원 뼈 프로필	표시 시험 후 방향·길이 검증
초기화 후 미세한 차이	누적 편집·원본 저장·수치 허용 오차	immutable 기준에서 재계산
다른 PC에서 실패	Node·모델·MPFB·Blender 경로	가이드와 별도 자산 전달 확인

원인 기록의 최소 단위
모델 해시, 변경한 한 값, 재현 순서, 앞/옆 캡처, 실제 관절 측정값을 함께 남깁니다. 화면 한 장만으로 스킨·뼈·재질 문제를 단정하지 않습니다.

검토 제안 · 기능 미구현

아직 만들지 않은 것

기존 검증 기능과 새 아이디어를 구분해서 다음 실험을 고릅니다.

자세 하나 저장
관절값 복사본
→
프레임 두 개
시각과 순서
→
중간 동작
보간·제한 검사
→
한 주기
경계·접지 검토

주제	현재 상태	다음 작은 검토
얼굴·가슴 크기	원본 변형 데이터는 존재 · 웹 미연결	한 항목 전달 + 길이 편집과 공존
다리 간격	고관절의 바깥 5° 고정	골반 너비/벌림 · 좌우·범위 정의
사용자 프레임	저장·타임라인·보간 미구현	자세 1개 저장·불러오기
영상 녹화	미구현	프레임 데이터 저장과 분리
실제 코스·기록	좌표·동작·계산 연결 미완료	검증할 단계와 지원 범위 구분

원본 외형 변형을 적용하고 길이·뼈대를 맞춘 다음 자세를 재생하는 순서는 설계 제안입니다. 현재 코드가 morph 데이터를 굳혀 제거한다는 점부터 함께 검토해야 합니다.

애니메이션이 생겨도 자동으로 되는 것은 아닙니다
프레임을 연결해 보기 좋은 움직임을 만드는 일, 발 접촉을 맞추는 일, 자세에 따른 기록을 계산하는 일은 각각 별도 검증이 필요합니다.

근거: docs/03-screens/MOTION_EDITOR.md

확인 질문과 해설

스스로 설명해 보기

답을 보기 전에 그림이나 한 줄 식으로 먼저 설명해 보세요.

질문	핵심 답
1. GLB를 불러오면 왜 달리지 않나요?	현재 출력은 animation을 끄며, 렌더 루프는 같은 자세도 반복해서 그립니다.
2. morph 슬라이더는 왜 cm가 아닌가요?	정점 변형의 가중치일 뿐, 측정점과 단위로 보정한 신체 치수가 아니기 때문입니다.
3. 길이와 자세를 왜 분리하나요?	길이는 기준 관절 위치·표면을 바꾸고, 자세는 그 기준에서의 회전을 바꾸기 때문입니다.
4. 부모의 역회전은 왜 곱하나요?	world에서 만든 목표 회전을 부모 기준의 local 회전으로 바꾸기 위해서입니다.
5. 매번 초기값으로 돌아가는 이유는?	이전 조작의 회전·변형이 누적되는 것을 막고 복원 기준을 유지하기 위해서입니다.
6. 작은 오차면 실제 인체도 정확한가요?	아닙니다. 코드의 기하학적 일관성 검사와 실측 신체·생체역학 검증은 다릅니다.
7. 원격 배포에 관리자 화면을 열면 되나요?	현재 로컬 제한으로 운영 인증을 대신할 수 없습니다. 서버 권한·게시·통계가 별도입니다.

학습 기록 제안

각 실습마다 **예상 → 조작 → 측정 → 차이 → 원인**을 한 줄씩 적습니다. 결과가 맞더라도 어떤 범위를 검사하지 않았는지 한 가지 함께 남겨보세요.

다음 회의의 출발점
저장 프레임 1개, 얼굴 크기 1개, 다리 벌림 1개 중 목적과 검수 기준을 가장 명확히 설명할 수 있는 작업부터 검토합니다. 이 교재가 새 기능 구현 승인을 대신하지 않습니다.

참고

용어와 코드 찾아가기

개념을 잊었을 때 정의와 실제 구현 위치를 함께 찾습니다.

용어	뜻	실제 위치
Rest pose	변형·회전을 시작하는 기준 자세	running-pose.ts / body-dimensions.ts
Skinning	뼈의 영향을 가중치로 섞어 표면을 변형	body-dimensions.ts
Morph / Shape key	기본 형상에 대한 정점 변화	export_runner.py / GLB
Quaternion	회전을 표현하는 네 성분 값	running-pose.ts: aim
Bind / inverse bind	뼈와 표면의 기준 연결·역변환	body-dimensions.ts: rebuild
Normal	표면의 조명 계산에 쓰는 방향	computeVertexNormals
Bounding box	물체를 감싸는 축 정렬 경계 상자	main.ts: frame / resizeBody
SHA-256	파일 바이트를 식별하는 해시	settings.ts / export scripts
Revision	관리자 설정의 변경 번호	settings.ts: saveSettings
Keyframe	특정 시각의 자세 등 상태	현재는 설계 문서만 존재

핵심 파일

근거: experiments/model-preview/src/main.ts

근거: experiments/model-preview/src/running-pose.ts

근거: experiments/model-preview/src/body-dimensions.ts

근거: experiments/model-preview/src/settings.ts

근거: experiments/model-preview/src/analytics.ts

근거: experiments/model-preview/tests/body-dimensions.cjs

출처 · 2026-10-14 확인

출처와 재현 범위

프로젝트의 실제 코드·기록을 우선하며 공식 문서는 개념 확인에 사용했습니다.

프로젝트 기준: 커밋 **de56d87595e8aa3f6f4b61bbec4169641b4ef2ca**. 코드 링크는 이 시점에 고정했습니다. 기술 버전은 최신 권장안이 아니라 당시 package.json·내보내기 기록을 설명합니다.

S1 · Three.js GLTFLoader - GLB 해석·비동기 로드·반환 자료

S2 · Three.js SkinnedMesh - 스킨·가중치·바인딩·경계

S3 · Three.js Quaternion - 회전·정규화·두 방향 사이 회전

S4 · MPFB Getting started - 설치·시작 참고

S5 · MPFB Export copy - 복사본·변형키·보조 형상

S6 · MDN IndexedDB API - 브라우저 저장소 개념

S7 · Playwright Assertions - 자동 검사 참고 · 이 프로젝트는 Node assert 사용

프로젝트 증거

근거: experiments/model-preview/public/models/runner-male.export.json

근거: experiments/model-preview/public/models/runner-female.export.json

근거: docs/05-validation/reports/body-dimensions-verification.json

근거: docs/05-validation/reports/admin-lab-verification.json

근거: docs/07-references/makehuman-assets.json

그림·수치의 출처
모든 화면은 저장소의 기존 실제 캡처이며 촬영 당시 UI입니다. 도식은 이 교재를 위해 작성한 개념도입니다. 24건 길이 검사와 수치는 기존 JSON을 읽어 넣었으며, 실제 사람·동작 성능에 대한 새로운 실험 결과가 아닙니다.

저작·재현: 교재 생성 원본은 같은 폴더의 build_textbook.py입니다. PDF에는 한글 폰트를 포함합니다. 모델·의류 등의 배포 권리는 개별 자산 출처에서 별도로 확인합니다.